TECNOLÓGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

ENERO- AGOSTO 2026

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

PATRONES DE DISEÑO DE SOFTWARE

UNIDAD A EVALUAR:

UNIDAD 3

DOCENTE:
MARIBEL GUERRERO LUIS

EQUIPO

CASTILLO LORENZO GERARDO - 22210290
DIAZ MORALES KATHERINE GISELLE - 22210302
DE HARO EXZACARIAS JULIO CESAR - 22210299
DEL RIO ROMERO LEONARDO JOEL - 22210301
RIVAS PEREZ KEVIN JONATHAN - 22210340

Tijuana Baja California, a 25 de marzo del 2026

PATRONES DE DISEÑO DECORADOR Y BRIDGE
Aplicado a restaurante abstracto

Patrón decorador

De acuerdo al patrón, cualquier objeto puede complementarse con un comportamiento deseado sin influir en las funcionalidades de otros objetos de la misma clase. El enfoque permite a los desarrolladores añadir y eliminar las dependencias de un objeto de manera dinámica y, cuando sea necesario, durante el tiempo de ejecución.

patrón Bridge

Permite dividir una clase grande, o un grupo de clases estrechamente relacionadas, en dos jerarquías separadas (abstracción e implementación) que pueden desarrollarse independientemente la una de la otra.

Una de estas jerarquías (a menudo denominada Abstracción) obtendrá una referencia a un objeto de la segunda jerarquía (Implementación). La abstracción podrá delegar algunas (en ocasiones, la mayoría) de sus llamadas al objeto de las implementaciones. Como todas las implementaciones tendrán una interfaz común, serán intercambiables dentro de la abstracción.

Objetivo

El objetivo es reutilizar el programa de restaurante abstracto e incluir los patrones decorador y bridge, agregando nuevas funciones de personalización como extras o comportamientos adicionales de manera flexible, que nos permita estructurar la lógica del negocio en capas y mejorar la experiencia del cliente.

Planteamiento del problema

Actualmente el sistema implementado del restaurante no está diseñado para poder combinar, agregar o excluir ingredientes o extras que los clientes desean en sus visitas, ya que el hacer combinaciones requiere crear nuevos registros y esto genera una gran cantidad de combinaciones y el patrón de diseño principal no lo soporta.

Buscamos de igual manera promocionar mas platillos, incluir promociones y con ello hacer que el restaurante atraiga mas clientela de la que está acostumbrada, para ello se tomará en cuenta lo siguiente:

- Agregar o quitar ingredientes de forma dinámica.
- Calcular el precio de los extras en cada plato, bebida o postre.
- Hacer flexible el código principal.
- Evitar sobre escribir el código y causar un problema de compilacion y que cree un error.
- Que sea de fácil comprensión y uso para el usuario así como descriptivo para cocina.
- Agregar descripción o observación sobre alergias en ingredientes en platillos.
- Agregar detalles de los ingredientes así como de su costo para evitar problemática de confusión por el cargo del plato.

El problema principal es que el patrón abstract factory no soporta cambios dinámicos en los objetos ya creados, es por eso que se necesitan implementar dos patrones adicionales para una mayor flexibilidad.

Código

En el patrón decorador tenemos que dentro del código se creó la clase abstracta PlatilloDecorador que envuelve un plato fuerte y permite agregar extras o ingredientes a los platillos sin modificar las clases originales como por ejemplo, guacamole, salsa de soya, parmesano, etc. También, cada decorador añade su costo y modifica la descripción al servir, y costo se incorpora en PlatoFuerte para calcular el precio con extras.

Por otro lado, dentro del patrón bridge se añadió IEstiloservicio con implementaciones concretas como ServicioMesa, ServicioParaLlevar y ServicioExpress. La clase abstracta de RestauranteBase recibe un IEstiloservicio por constructor para separar el restaurante del tipo de servicio como mesa, para llevar y express sin modificar las clases de los restaurantes.

cliente.cs

```
using FabricasRestaurantes;
using System;

namespace FabricasRestaurantes
{
    internal class FabricaRestaurantes
    {
        public static void ProbarConsola()
        {
            RestauranteFactory fabrica;


            fabrica = new RestauranteChino();
            PlatoFuerte plato = fabrica.CrearPlatoFuerte();
            Bebida bebida = fabrica.CrearBebida();
            Postre postre = fabrica.CrearPostre();
            Console.WriteLine($"Plato:  {plato.Servir()}");
            Console.WriteLine($"Bebida: {bebida.Servir()}");
            Console.WriteLine($"Postre: {postre.Servir()}");
            Console.WriteLine();


            fabrica = new RestauranteJapones();
            plato = fabrica.CrearPlatoFuerte();
            bebida = fabrica.CrearBebida();
            postre = fabrica.CrearPostre();
            Console.WriteLine($"Plato:  {plato.Servir()}");
            Console.WriteLine($"Bebida: {bebida.Servir()}");
            Console.WriteLine($"Postre: {postre.Servir()}");
            Console.WriteLine();
```

```
            fabrica = new RestauranteMexicano();
            plato = fabrica.CrearPlatoFuerte();
            bebida = fabrica.CrearBebida();
            postre = fabrica.CrearPostre();
            Console.WriteLine($"Plato:  {plato.Servir()}");
            Console.WriteLine($"Bebida: {bebida.Servir()}");
            Console.WriteLine($"Postre: {postre.Servir()}");
            Console.WriteLine();

            fabrica = new RestauranteItaliano();
            plato = fabrica.CrearPlatoFuerte();
            bebida = fabrica.CrearBebida();
            postre = fabrica.CrearPostre();
            Console.WriteLine($"Plato:  {plato.Servir()}");
            Console.WriteLine($"Bebida: {bebida.Servir()}");
            Console.WriteLine($"Postre: {postre.Servir()}");
            Console.WriteLine();


            fabrica = new RestauranteFrances();
            plato = fabrica.CrearPlatoFuerte();
            bebida = fabrica.CrearBebida();
            postre = fabrica.CrearPostre();
            Console.WriteLine($"Plato:  {plato.Servir()}");
            Console.WriteLine($"Bebida: {bebida.Servir()}");
            Console.WriteLine($"Postre: {postre.Servir()}");
            Console.WriteLine();

            Console.ReadKey();
        }
    }
}
```

RestauranteFactory.cs

```
namespace FabricasRestaurantes
{

    public abstract class RestauranteFactory
    {
        public abstract PlatoFuerte CrearPlatoFuerte();
        public abstract Bebida CrearBebida();
        public abstract Postre CrearPostre();
    }


    public abstract class PlatoFuerte
    {
        public abstract string Servir();
        public abstract double Costo { get; }
    }

    public abstract class Bebida
    {
        public abstract string Servir();
    }

    public abstract class Postre
```

```
    {
        public abstract string Servir();
    }
}
```

RestauranteChino.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteChino : RestauranteFactory
    {
        public override PlatoFuerte CrearPlatoFuerte() => new ChowMein();
        public override Bebida CrearBebida() => new TeJazmin();
        public override Postre CrearPostre() => new RollitoDulce();
    }

    class ChowMein : PlatoFuerte
    {
        public override string Servir() => "Chow Mein";
        public override double Costo => 85;
    }

    class TeJazmin : Bebida
    {
        public override string Servir() => "Té Jazmín";
    }

    class RollitoDulce : Postre
    {
        public override string Servir() => "Rollito dulce con nieve";
    }

    public class SalsaSoya : PlatilloDecorador
    {
        public SalsaSoya(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 10;
        public override string Servir() => $"{_platillo.Servir()}, Salsa Soya";
    }
    public class Pollo : PlatilloDecorador
    {
        public Pollo(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 40;
        public override string Servir() => $"{_platillo.Servir()}, Pollo";
    }
    public class Camaron : PlatilloDecorador
    {
        public Camaron(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 60;
        public override string Servir() => $"{_platillo.Servir()}, Camaron";
    }
}
```

RestauranteJapones.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteJapones : RestauranteFactory
    {
```

```
        public override PlatoFuerte CrearPlatoFuerte() => new Ramen();
        public override Bebida CrearBebida() => new Sake();
        public override Postre CrearPostre() => new Dango();
    }

    class Ramen : PlatoFuerte
    {
        public override string Servir() => "Ramen";
        public override double Costo => 90;
    }

    class Sake : Bebida
    {
        public override string Servir() => "Sake";
    }

    class Dango : Postre
    {
        public override string Servir() => "Dango";
    }

    public class HuevoCocido : PlatilloDecorador
    {
        public HuevoCocido(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 10;
        public override string Servir() => $"{_platillo.Servir()}, HuevoCocido";
    }
    public class ChasuPork : PlatilloDecorador
    {
        public ChasuPork(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 25;
        public override string Servir() => $"{_platillo.Servir()}, ChasuPork";
    }
    public class Nori : PlatilloDecorador
    {
        public Nori(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 8;
        public override string Servir() => $"{_platillo.Servir()}, Nori";
    }
}
```

RestauranteMexicano.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteMexicano : RestauranteFactory
    {
        public override PlatoFuerte CrearPlatoFuerte() => new TacosCarneAsada();
        public override Bebida CrearBebida() => new AguaJamaica();
        public override Postre CrearPostre() => new PastelTresLeches();
    }

    class TacosCarneAsada : PlatoFuerte
    {
        public override string Servir() => "Tacos de carne asada";
        public override double Costo => 70;
    }
```

```
    class AguaJamaica : Bebida
    {
        public override string Servir() => "Agua de Jamaica";
    }

    class PastelTresLeches : Postre
    {
        public override string Servir() => "Pastel de tres leches";
    }
    public class Guacamole : PlatilloDecorador
    {
        public Guacamole(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 15;
        public override string Servir() => $"{_platillo.Servir()}, Guacamole";
    }
    public class ChileGuero : PlatilloDecorador
    {
        public ChileGuero(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 10;
        public override string Servir() => $"{_platillo.Servir()}, Chiles Gueros a la parrilla";
    }
    public class SalsaRoja : PlatilloDecorador
    {
        public SalsaRoja(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 5;
        public override string Servir() => $"{_platillo.Servir()}, Salsa Roja";
    }
}
```

RestauranteItaliano.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteItaliano : RestauranteFactory
    {
        public override PlatoFuerte CrearPlatoFuerte() => new Pasta();
        public override Bebida CrearBebida() => new VinoTinto();
        public override Postre CrearPostre() => new Tiramisu();
    }

    class Pasta : PlatoFuerte
    {
        public override string Servir() => "Pasta";
        public override double Costo => 95;
    }

    class VinoTinto : Bebida
    {
        public override string Servir() => "Vino tinto";
    }

    class Tiramisu : Postre
    {
        public override string Servir() => "Tiramisú";
    }

    public class QuesoParmesano : PlatilloDecorador
    {
```

```
        public QuesoParmesano(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 18;
        public override string Servir() => $"{_platillo.Servir()}, Queso parmesano";
    }
    public class Albahaca : PlatilloDecorador
    {
        public Albahaca(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 7;
        public override string Servir() => $"{_platillo.Servir()}, Albahaca";
    }
    public class PepperoniExtra : PlatilloDecorador
    {
        public PepperoniExtra(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 22;
        public override string Servir() => $"{_platillo.Servir()}, Pepperoni extra";
    }
}
```

RestauranteFrances.cs

```
namespace FabricasRestaurantes
{
    public class RestauranteFrances : RestauranteFactory
    {
        public override PlatoFuerte CrearPlatoFuerte() => new CoqAuVin();
        public override Bebida CrearBebida() => new Champan();
        public override Postre CrearPostre() => new CremeBrulee();
    }

    class CoqAuVin : PlatoFuerte
    {
        public override string Servir() => "Coq au Vin";
        public override double Costo => 130;
    }

    class Champan : Bebida
    {
        public override string Servir() => "Champán";
    }

    class CremeBrulee : Postre
    {
        public override string Servir() => "Crème Brûlée";
    }

    public class Champis : PlatilloDecorador
    {
        public Champis(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 20;
        public override string Servir() => $"{_platillo.Servir()}, Champiñones";
    }
    public class HierbasFinas : PlatilloDecorador
    {
        public HierbasFinas(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 12;
        public override string Servir() => $"{_platillo.Servir()}, HierbasFinas";
    }
    public class SalsaVino : PlatilloDecorador
```

```
    {
        public SalsaVino(PlatoFuerte platillo) : base(platillo) { }
        public override double Costo => _platillo.Costo + 28;
        public override string Servir() => $"{_platillo.Servir()}, Salsa de vino";
    }
}
```

RestauranteBase.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FabricasRestaurantes
{
    public abstract class RestauranteBase
    {
        protected IEstiloservicio _estiloServicio;

        public RestauranteBase(IEstiloservicio estiloServicio)
        {
            _estiloServicio = estiloServicio;
        }

        public string PreparararMesa()
        {
            return _estiloServicio.PreparararMesa();
        }

        public string ServirPlatillo()
        {
            return _estiloServicio.ServirPlatillo();
        }

        public string CobrarCuenta()
        {
            return _estiloServicio.CobrarCuenta();
        }


        public void AsignarServicio(IEstiloservicio estiloServicio)
        {
            _estiloServicio = estiloServicio;
        }

        public IEstiloservicio ObtenerServicio()
        {
            return _estiloServicio;
        }
    }
}
```

PlatilloDecorador.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FabricasRestaurantes
{
    public abstract class PlatilloDecorador : PlatoFuerte
    {
        protected PlatoFuerte _platillo;

        public PlatilloDecorador(PlatoFuerte platillo)
        {
            _platillo = platillo;
        }

        public override string Servir() => _platillo.Servir();
        public override double Costo => _platillo.Costo;
    }
}
```

Meserorestaurante.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FabricasRestaurantes
{
    class MeseroRestaurante : RestauranteBase
    {
        private string _nombreRestaurante;

        public MeseroRestaurante(string nombreRestaurante) : base(new ServicioMesa())
        {
            _nombreRestaurante = nombreRestaurante;
        }


        public MeseroRestaurante(string nombreRestaurante, IEstiloservicio estiloServicio) : base(estiloServicio)
        {
            _nombreRestaurante = nombreRestaurante;
        }

        public string NombreRestaurante()
        {
            return "Atendiendo en: Restaurante " + _nombreRestaurante;
        }
    }
}
```

IEstiloservicio.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FabricasRestaurantes
{
    public interface IEstiloservicio
    {
        string PrepararMesa();
        string ServirPlatillo();
        string CobrarCuenta();
    }
}
```

Estilosservicio.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FabricasRestaurantes
{
    class ServicioMesa : IEstiloservicio
    {
        public string PrepararMesa() => "Mesa preparada con mantel y cubiertos";
        public string ServirPlatillo() => "Platillo servido en mesa por el mesero";
        public string CobrarCuenta() => "Cuenta presentada en mesa";
    }
}

namespace FabricasRestaurantes
{
    class ServicioParaLlevar : IEstiloservicio
    {
        public string PrepararMesa() => "Empaque preparado para llevar";
        public string ServirPlatillo() => "Platillo empacado en contenedor";
        public string CobrarCuenta() => "Pago en caja antes de retirar";
    }
}

namespace FabricasRestaurantes
{
    class ServicioExpress : IEstiloservicio
    {
        public string PrepararMesa() => "Pedido registrado para entrega express";
        public string ServirPlatillo() => "Platillo entregado en puerta";
        public string CobrarCuenta() => "Pago procesado en línea";
    }
```

```
}
```

MainForm.cs

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;

namespace FabricasRestaurantes
{
    public class MainForm : Form
    {
        private readonly string[] _nombres = { "Chino", "Japonés", "Mexicano", "Italiano", "Francés" };
        private readonly string[] _emojis = { "🥢", "🍱", "🌮", "🍝", "🥐" };
        private readonly Color[] _primarios =
        {
            Color.FromArgb(190, 30,  45),
            Color.FromArgb( 20, 60, 120),
            Color.FromArgb(200, 80,   0),
            Color.FromArgb(180, 20,  20),
            Color.FromArgb( 25, 50, 125),
        };
        private readonly Color[] _acentos =
        {
            Color.FromArgb(255, 220, 120),
            Color.FromArgb(255, 182, 193),
            Color.FromArgb(255, 220,  50),
            Color.FromArgb(245, 245, 200),
            Color.FromArgb(255, 235, 160),
        };

        private readonly RestauranteFactory[] _fabricas =
        {
            new RestauranteChino(),
            new RestauranteJapones(),
            new RestauranteMexicano(),
            new RestauranteItaliano(),
            new RestauranteFrances(),
        };

        private readonly double[] _costoBase = { 85, 90, 70, 95, 130 };
        private readonly double[] _costoBebida = { 30, 25, 20, 35, 40 };
        private readonly double[] _costoPostre = { 25, 30, 15, 28, 45 };

        private int _idx = 0;

        private Panel _sidebar = new Panel();
        private Panel _pnlCard = new Panel();
        private Panel _pnlHeader = new Panel();
        private Panel _pnlAgregados = new Panel();
        private Label _lblTitulo = new Label();
        private Label _lblSubtitulo = new Label();
```

```
private Label _lblPlato = new Label();
private Label _lblBebida = new Label();
private Label _lblPostre = new Label();
private Button[] _botones = new Button[5];


private PlatoFuerte _platilloActual = null!;
private double _costoExtra = 0;

public MainForm()
{
    InitializeComponent();
    CrearBotones();
    CrearContenidoCard();
    AjustarLayout();
    MostrarSelectorServicio(0);
}

private void InitializeComponent()
{
    this.Text = "Fábrica de Restaurantes";
    this.Size = new Size(860, 660);
    this.MinimumSize = new Size(760, 600);
    this.StartPosition = FormStartPosition.CenterScreen;
    this.BackColor = Color.FromArgb(22, 22, 32);
    this.Font = new Font("Segoe UI", 10f);
    this.DoubleBuffered = true;


    _sidebar.Dock = DockStyle.Left;
    _sidebar.Width = 210;
    _sidebar.BackColor = Color.FromArgb(30, 30, 42);

    Label lblLogo = new Label();
    lblLogo.Text = "🍽 RESTAURANTES";
    lblLogo.Font = new Font("Segoe UI", 10f, FontStyle.Bold);
    lblLogo.ForeColor = Color.FromArgb(160, 160, 200);
    lblLogo.AutoSize = false;
    lblLogo.Bounds = new Rectangle(0, 16, 210, 36);
    lblLogo.TextAlign = ContentAlignment.MiddleCenter;
    lblLogo.BackColor = Color.Transparent;
    _sidebar.Controls.Add(lblLogo);

    Panel sep = new Panel();
    sep.Bounds = new Rectangle(14, 58, 182, 1);
    sep.BackColor = Color.FromArgb(55, 55, 75);
    _sidebar.Controls.Add(sep);

    Label lblCredito = new Label();
    lblCredito.Text = "Abstract Factory + Decorator";
    lblCredito.Font = new Font("Segoe UI", 7.5f, FontStyle.Italic);
    lblCredito.ForeColor = Color.FromArgb(70, 70, 95);
    lblCredito.TextAlign = ContentAlignment.MiddleCenter;
    lblCredito.Dock = DockStyle.Bottom;
    lblCredito.Height = 28;
    lblCredito.BackColor = Color.Transparent;
    _sidebar.Controls.Add(lblCredito);
```

```
    Panel pnlMain = new Panel();
    pnlMain.Dock = DockStyle.Fill;
    pnlMain.BackColor = Color.FromArgb(22, 22, 32);

    _pnlHeader.BackColor = Color.Transparent;
    _pnlHeader.Bounds = new Rectangle(28, 20, 580, 90);
    _pnlHeader.Paint += PnlHeader_Paint;

    _lblTitulo.Font = new Font("Segoe UI", 22f, FontStyle.Bold);
    _lblTitulo.ForeColor = Color.White;
    _lblTitulo.AutoSize = false;
    _lblTitulo.Bounds = new Rectangle(0, 8, 580, 46);
    _lblTitulo.BackColor = Color.Transparent;

    _lblSubtitulo.Font = new Font("Segoe UI", 10f);
    _lblSubtitulo.ForeColor = Color.FromArgb(140, 140, 175);
    _lblSubtitulo.AutoSize = false;
    _lblSubtitulo.Bounds = new Rectangle(2, 54, 580, 26);
    _lblSubtitulo.BackColor = Color.Transparent;

    _pnlHeader.Controls.Add(_lblTitulo);
    _pnlHeader.Controls.Add(_lblSubtitulo);

    _pnlCard.BackColor = Color.FromArgb(35, 35, 50);
    _pnlCard.Bounds = new Rectangle(28, 126, 580, 300);
    _pnlCard.Paint += PnlCard_Paint;


    _pnlAgregados.BackColor = Color.FromArgb(28, 28, 42);
    _pnlAgregados.Bounds = new Rectangle(28, 440, 580, 150);
    _pnlAgregados.Visible = false;

    pnlMain.Controls.Add(_pnlHeader);
    pnlMain.Controls.Add(_pnlCard);
    pnlMain.Controls.Add(_pnlAgregados);

    this.Controls.Add(pnlMain);
    this.Controls.Add(_sidebar);

    this.Resize += MainForm_Resize;
}


// PASO 1 — Agregados del platillo

private void MostrarPasoPlatillo()
{
    _costoExtra = 0;
    _pnlAgregados.Controls.Clear();
    _pnlAgregados.Visible = true;

    Label lbl = new Label
    {
        Text = "✨ Personaliza tu platillo:",
        ForeColor = _acentos[_idx],
        Font = new Font("Segoe UI", 9.5f, FontStyle.Bold),
        AutoSize = false,
```

```
        Bounds = new Rectangle(12, 8, 350, 24),
        BackColor = Color.Transparent,
    };
    _pnlAgregados.Controls.Add(lbl);

    var agregados = ObtenerAgregadosPlatillo(_idx);
    for (int i = 0; i < agregados.Count; i++)
    {
        var chk = new CheckBox
        {
            Text = agregados[i].Etiqueta,
            ForeColor = Color.FromArgb(210, 210, 230),
            Font = new Font("Segoe UI", 9.5f),
            AutoSize = false,
            Bounds = new Rectangle(12 + i * 190, 38, 185, 26),
            BackColor = Color.Transparent,
            Tag = agregados[i].Crear,
        };
        _pnlAgregados.Controls.Add(chk);
    }

    Button btnSig = CrearBoton("Siguiente →", new Rectangle(12, 80, 150, 36));
    btnSig.Click += (s, e) =>
    {

        _platilloActual = _fabricas[_idx].CrearPlatoFuerte();

        foreach (Control ctrl in _pnlAgregados.Controls)
            if (ctrl is CheckBox chk && chk.Checked
                && chk.Tag is Func<PlatoFuerte, PlatilloDecorador> crear)
                _platilloActual = crear(_platilloActual);

        MostrarPasoBebida();
    };
    _pnlAgregados.Controls.Add(btnSig);
}


//Tamaño de bebida

private string _tamanoBebida = "Chico";
private MeseroRestaurante _meseroActual = null!;
private string _estiloServicio = "Mesa";
private void MostrarPasoBebida()
{
    _pnlAgregados.Controls.Clear();

    Label lbl = new Label
    {
        Text = " 🥤 Personaliza tu bebida:",
        ForeColor = _acentos[_idx],
        Font = new Font("Segoe UI", 9.5f, FontStyle.Bold),
        AutoSize = false,
        Bounds = new Rectangle(12, 8, 350, 24),
        BackColor = Color.Transparent,
    };
    _pnlAgregados.Controls.Add(lbl);
```

```
    var tamanios = new (string Texto, double Costo)[]
    {
        ("Chico   +$0",   0),
        ("Mediano +$10", 10),
        ("Grande  +$20", 20),
    };

    var radios = new RadioButton[3];
    for (int i = 0; i < tamanios.Length; i++)
    {
        var rb = new RadioButton
        {
            Text = tamanios[i].Texto,
            ForeColor = Color.FromArgb(210, 210, 230),
            Font = new Font("Segoe UI", 9.5f),
            AutoSize = false,
            Bounds = new Rectangle(12 + i * 160, 38, 155, 26),
            BackColor = Color.Transparent,
            Tag = tamanios[i].Costo,
        };
        if (i == 0) rb.Checked = true; ///
        _pnlAgregados.Controls.Add(rb);
        radios[i] = rb;
    }


    Button btnSig = CrearBoton("Siguiente →", new Rectangle(12, 80, 150, 36));
    btnSig.Click += (s, e) =>
    {
        foreach (var rb in radios)
            if (rb.Checked && rb.Tag is double costo)
            {
                _costoExtra += costo;
                _tamanoBebida = rb.Text.Split(' ')[0];
            }
        MostrarPasoPostre();
    };
    _pnlAgregados.Controls.Add(btnSig);
}
// Porción extra del postre

private void MostrarPasoPostre()
{
    _pnlAgregados.Controls.Clear();

    Label lbl = new Label
    {
        Text = "🍮 Personaliza tu postre:",
        ForeColor = _acentos[_idx],
        Font = new Font("Segoe UI", 9.5f, FontStyle.Bold),
        AutoSize = false,
        Bounds = new Rectangle(12, 8, 350, 24),
        BackColor = Color.Transparent,
    };
    _pnlAgregados.Controls.Add(lbl);

    var chkExtra = new CheckBox
    {
```

```
        Text = "Porción extra  +$15",
        ForeColor = Color.FromArgb(210, 210, 230),
        Font = new Font("Segoe UI", 9.5f),
        AutoSize = false,
        Bounds = new Rectangle(12, 38, 200, 26),
        BackColor = Color.Transparent,
    };
    _pnlAgregados.Controls.Add(chkExtra);

    Button btnConfirmar = CrearBoton("Confirmar pedido", new Rectangle(12, 80, 165, 36));
    btnConfirmar.Click += (s, e) =>
    {
        if (chkExtra.Checked) _costoExtra += 15;


        double total = _platilloActual.Costo + _costoBebida[_idx] + _costoPostre[_idx] + _costoExtra;


        string postreTexto = _fabricas[_idx].CrearPostre().Servir()
                            + (chkExtra.Checked ? " (porción extra)" : "");

        MessageBox.Show(
            $"🪑 {_meseroActual.NombreRestaurante()}\n" +
            $"   • {_meseroActual.PrepararMesa()}\n" +
            $"   • {_meseroActual.ServirPlatillo()}\n" +
            $"   • {_meseroActual.CobrarCuenta()}\n\n" +
            $"🍽️ {_platilloActual.Servir()}\n" +
            $"🥤 {_fabricas[_idx].CrearBebida().Servir()} ({_tamanoBebida})\n" +
            $"🍮 {postreTexto}\n\n" +
            $"💰 Costo total: ${total}",
            "Pedido confirmado ✅",
            MessageBoxButtons.OK,
            MessageBoxIcon.Information
        );

        _pnlAgregados.Visible = false;
    };
    _pnlAgregados.Controls.Add(btnConfirmar);
}


private Button CrearBoton(string texto, Rectangle bounds)
{
    var btn = new Button
    {
        Text = texto,
        BackColor = _primarios[_idx],
        ForeColor = Color.White,
        FlatStyle = FlatStyle.Flat,
        Font = new Font("Segoe UI", 9.5f, FontStyle.Bold),
        Bounds = bounds,
        Cursor = Cursors.Hand,
    };
    btn.FlatAppearance.BorderSize = 0;
    return btn;
}
```

```
    private List<(string Etiqueta, Func<PlatoFuerte, PlatilloDecorador> Crear)> ObtenerAgregadosPlatillo(int
idx)
    {
        var lista = new List<(string, Func<PlatoFuerte, PlatilloDecorador>)>();
        switch (idx)
        {
            case 0:
                lista.Add(("Salsa Soya  +$10", (PlatoFuerte p) => new SalsaSoya(p)));
                lista.Add(("Pollo       +$20", (PlatoFuerte p) => new Pollo(p)));
                lista.Add(("Camarón     +$30", (PlatoFuerte p) => new Camaron(p)));
                break;
            case 1:
                lista.Add(("Huevo Cocido  +$12", (PlatoFuerte p) => new HuevoCocido(p)));
                lista.Add(("Chasu Pork    +$25", (PlatoFuerte p) => new ChasuPork(p)));
                lista.Add(("Nori          +$8", (PlatoFuerte p) => new Nori(p)));
                break;
            case 2:
                lista.Add(("Guacamole    +$15", (PlatoFuerte p) => new Guacamole(p)));
                lista.Add(("ChileGuero +$10", (PlatoFuerte p) => new ChileGuero(p)));
                lista.Add(("Salsa Roja   +$5", (PlatoFuerte p) => new SalsaRoja(p)));
                break;
            case 3:
                lista.Add(("Queso Parmesano +$18", (PlatoFuerte p) => new QuesoParmesano(p)));
                lista.Add(("Albahaca        +$7", (PlatoFuerte p) => new Albahaca(p)));
                lista.Add(("Pepperoni Extra +$22", (PlatoFuerte p) => new PepperoniExtra(p)));
                break;
            case 4:
                lista.Add(("Champiñones   +$20", (PlatoFuerte p) => new Champis(p)));
                lista.Add(("Hierbas Finas +$12", (PlatoFuerte p) => new HierbasFinas(p)));
                lista.Add(("Salsa de Vino +$28", (PlatoFuerte p) => new SalsaVino(p)));
                break;
        }
        return lista;
    }

    private void MainForm_Resize(object? sender, EventArgs e) => AjustarLayout();

    private void CrearBotones()
    {
        for (int i = 0; i < 5; i++)
        {
            int idx = i;
            var btn = new Button();
            btn.Text = _emojis[i] + "  Restaurante " + _nombres[i];
            btn.TextAlign = ContentAlignment.MiddleLeft;
            btn.Font = new Font("Segoe UI", 10f);
            btn.ForeColor = Color.FromArgb(190, 190, 220);
            btn.BackColor = Color.FromArgb(38, 38, 54);
            btn.FlatStyle = FlatStyle.Flat;
            btn.Bounds = new Rectangle(10, 70 + i * 72, 190, 58);
            btn.Cursor = Cursors.Hand;
            btn.Padding = new Padding(10, 0, 0, 0);
            btn.FlatAppearance.BorderSize = 0;
            btn.FlatAppearance.MouseOverBackColor = Color.FromArgb(55, 55, 80);
            btn.Click += (s, e) => MostrarSelectorServicio(idx);
            _botones[i] = btn;
            _sidebar.Controls.Add(btn);
        }
```

```
}

private void CrearContenidoCard()
{
    string[] iconos = { "🍽️", "🥤", "🍮" };
    Label[] lbArr = new Label[3];
    int[] yPos = { 42, 142, 242 };

    for (int i = 0; i < 3; i++)
    {
        var ic = new Label();
        ic.Text = iconos[i];
        ic.Font = new Font("Segoe UI Emoji", 20f);
        ic.AutoSize = false;
        ic.Bounds = new Rectangle(24, yPos[i] - 2, 42, 42);
        ic.TextAlign = ContentAlignment.MiddleCenter;
        ic.BackColor = Color.Transparent;
        ic.ForeColor = Color.White;

        var lbl = new Label();
        lbl.Text = "";
        lbl.Font = new Font("Segoe UI", 11f);
        lbl.ForeColor = Color.White;
        lbl.AutoSize = false;
        lbl.Bounds = new Rectangle(76, yPos[i], 460, 42);
        lbl.TextAlign = ContentAlignment.MiddleLeft;
        lbl.BackColor = Color.Transparent;


        if (i == 0)
        {
            lbl.Cursor = Cursors.Hand;
            lbl.Font = new Font("Segoe UI", 11f, FontStyle.Underline);
            lbl.Click += (s, e) => MostrarPasoPlatillo();
        }

        _pnlCard.Controls.Add(ic);
        _pnlCard.Controls.Add(lbl);
        lbArr[i] = lbl;
    }

    _lblPlato = lbArr[0];
    _lblBebida = lbArr[1];
    _lblPostre = lbArr[2];
}

private void AjustarLayout()
{
    if (DesignMode) return;
    int w = Math.Max(this.ClientSize.Width - 210 - 56, 200);
    _pnlHeader.Width = w;
    _lblTitulo.Width = w;
    _lblSubtitulo.Width = w;
    _pnlCard.Width = w;
    _pnlAgregados.Width = w;
}

private void MostrarSelectorServicio(int idx)
```

```
{
    _idx = idx;
    _pnlAgregados.Controls.Clear();
    _pnlAgregados.Visible = true;
    _pnlCard.Visible = false;

    for (int i = 0; i < 5; i++)
    {
        if (i == idx)
        {
            _botones[i].BackColor = MezclarColor(_primarios[i], Color.FromArgb(38, 38, 54), 0.45f);
            _botones[i].ForeColor = _acentos[i];
            _botones[i].Font = new Font("Segoe UI", 10f, FontStyle.Bold);
        }
        else
        {
            _botones[i].BackColor = Color.FromArgb(38, 38, 54);
            _botones[i].ForeColor = Color.FromArgb(190, 190, 220);
            _botones[i].Font = new Font("Segoe UI", 10f);
        }
    }

    Label lbl = new Label
    {
        Text = "🪑 ¿Cómo deseas tu servicio?",
        ForeColor = _acentos[idx],
        Font = new Font("Segoe UI", 9.5f, FontStyle.Bold),
        AutoSize = false,
        Bounds = new Rectangle(12, 8, 350, 24),
        BackColor = Color.Transparent,
    };
    _pnlAgregados.Controls.Add(lbl);

    var estilos = new (string Texto, IEstiloservicio Servicio)[]
    {
        ("🍽️ En Mesa",      new ServicioMesa()),
        ("🛍️ Para Llevar",  new ServicioParaLlevar()),
        ("⚡ Express",       new ServicioExpress()),
    };

    var radios = new RadioButton[3];
    for (int i = 0; i < estilos.Length; i++)
    {
        var rb = new RadioButton
        {
            Text = estilos[i].Texto,
            ForeColor = Color.FromArgb(210, 210, 230),
            Font = new Font("Segoe UI", 9.5f),
            AutoSize = false,
            Bounds = new Rectangle(12 + i * 175, 38, 170, 26),
            BackColor = Color.Transparent,
            Tag = estilos[i].Servicio,
        };
        if (i == 0) rb.Checked = true;
        _pnlAgregados.Controls.Add(rb);
        radios[i] = rb;
    }
```

```
    Button btnSig = CrearBoton("Ver Menú →", new Rectangle(12, 80, 150, 36));
    btnSig.Click += (s, e) =>
    {
        foreach (var rb in radios)
        {
            if (rb.Checked && rb.Tag is IEstiloservicio estilo)
            {

                _meseroActual = new MeseroRestaurante(
                    _nombres[idx], estilo);
                _estiloServicio = rb.Text;
            }
        }

        _pnlCard.Visible = true;
        MostrarMenu(idx);
    };
    _pnlAgregados.Controls.Add(btnSig);
}

private void MostrarMenu(int idx)
{
    _idx = idx;
    var plato = _fabricas[idx].CrearPlatoFuerte();
    var bebida = _fabricas[idx].CrearBebida();
    var postre = _fabricas[idx].CrearPostre();

    _lblTitulo.Text = _emojis[idx] + "  Restaurante " + _nombres[idx];
    _lblTitulo.ForeColor = _acentos[idx];
    _lblSubtitulo.Text = "Menú del día  ·  cocina " + _nombres[idx].ToLower();
    _lblPlato.Text = plato.Servir() + $"  —  ${_costoBase[idx]}";
    _lblBebida.Text = bebida.Servir() + $"  —  ${_costoBebida[idx]}";
    _lblPostre.Text = postre.Servir() + $"  —  ${_costoPostre[idx]}";

    _pnlCard.BackColor = MezclarColor(_primarios[idx], Color.FromArgb(35, 35, 50), 0.15f);
    _pnlCard.Visible = true;

    _pnlCard.Invalidate();
    _pnlHeader.Invalidate();
}

private void PnlHeader_Paint(object? sender, PaintEventArgs e)
{
    if (DesignMode || !this.IsHandleCreated) return;
    var g = e.Graphics;
    g.SmoothingMode = SmoothingMode.AntiAlias;
    int w = _pnlHeader.Width;
    using (var brush = new LinearGradientBrush(
        new Point(0, 0), new Point(Math.Min(w, 340), 0),
        _primarios[_idx], Color.Transparent))
    {
        g.FillRectangle(brush, 0, _pnlHeader.Height - 3, Math.Min(w, 340), 3);
    }
}

private void PnlCard_Paint(object? sender, PaintEventArgs e)
{
    if (DesignMode || !this.IsHandleCreated) return;
```

```
        var g = e.Graphics;
        Panel card = (Panel)sender!;
        g.SmoothingMode = SmoothingMode.AntiAlias;

        using (var path = RoundedRect(new Rectangle(0, 0, card.Width - 1, card.Height - 1), 14))
        using (var bgBr = new SolidBrush(card.BackColor))
            g.FillPath(bgBr, path);

        using (var penSep = new Pen(Color.FromArgb(55, 55, 75), 1))
        {
            g.DrawLine(penSep, 20, 100, card.Width - 20, 100);
            g.DrawLine(penSep, 20, 200, card.Width - 20, 200);
        }

        using (var barBrush = new LinearGradientBrush(
            new Point(0, 16), new Point(0, card.Height - 16),
            _primarios[_idx], _acentos[_idx]))
        {
            g.FillRectangle(barBrush, 0, 16, 5, card.Height - 32);
        }
    }

    private static Color MezclarColor(Color a, Color b, float t) =>
        Color.FromArgb(
            (int)(a.R * t + b.R * (1 - t)),
            (int)(a.G * t + b.G * (1 - t)),
            (int)(a.B * t + b.B * (1 - t)));

    private static GraphicsPath RoundedRect(Rectangle r, int radius)
    {
        var p = new GraphicsPath();
        p.AddArc(r.X, r.Y, radius * 2, radius * 2, 180, 90);
        p.AddArc(r.Right - radius * 2, r.Y, radius * 2, radius * 2, 270, 90);
        p.AddArc(r.Right - radius * 2, r.Bottom - radius * 2, radius * 2, radius * 2, 0, 90);
        p.AddArc(r.X, r.Bottom - radius * 2, radius * 2, radius * 2, 90, 90);
        p.CloseFigure();
        return p;
    }
  }
}
```

Conclusión

Con este trabajo quedó demostrado que se puede integrar los patrones decorador y bridge al sistema de restaurantes, y como resultado se obtuvo un programa más flexible. Al final se comprende que los patrones no compiten, sino que más bien se complementan y dentro de esta práctica se pudo observar que abstract factory organiza las familias de productos, decorador agrega comportamientos variables y bridge separa responsabilidades. Con esto se reforzó la comprensión de cómo los patrones pueden trabajar juntos para resolver problemas complejos.

Referencias

https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/que-es-el-patron-de-diseno-decorator/

https://refactoring.guru/es/design-patterns/bridge#:~:text=Bridge%20es%20un%20patr%C3%B3n%20de,la%20una%20de%20la%20otra.

https://refactoring.guru/es/design-patterns/bridge/csharp/example